

Section 2: BRANCHING STRUCTURES

Lecture 8: Logical Expressions

Logical variables (LOGICAL :: a, b, c)

Logical constants (.TRUE. or .FALSE)

Relational expressions

<	.LT.	Is less than
>	.GT.	Is greater than
==	.EQ.	Is equal to
<=	.LE.	Less than or equal to
>=	.GE.	Greater than or equal to
/=	.NE.	Not equal to

Examples:

Y=100.

x=50.

y > x this logical expression is true.

10==20/2 this logical expression is true

What is the output from the following:

print*, 10==20/2

'G' < 'F' this logical expression is false

'G' .GE. 'GA' this logical expression is false

10/3 .EQ. 3 this logical expression is true

10./3 .EQ. 3 this logical expression is false

Compound the Expressions

.AND.

.OR.

For example, if:

```
X = 5.;      y=8.  
y > x .AND. y < 10      is true  
y < x .OR.  y > 10      is false  
y < x .OR.  y==8        is true
```

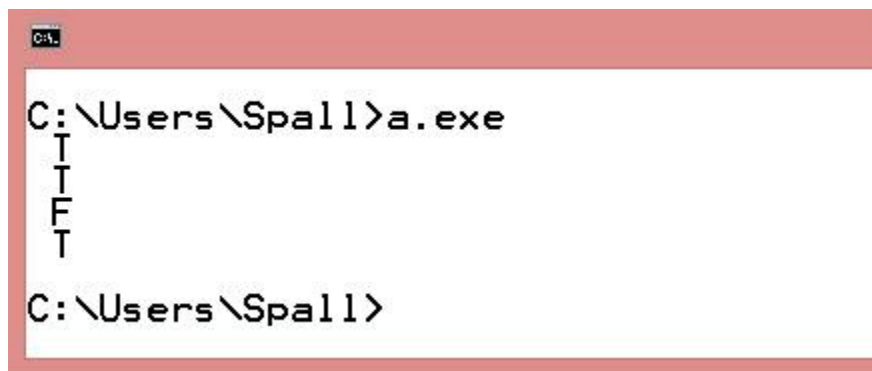
If:

X=6

Y=8

then what is the output of the following:

```
PRINT*, x==6 .OR. y==7  
PRINT*, x==6 .AND. y==8 .AND. x==8 .OR. y==8  
Print*, x==6 .and. y==7 .or. x==5 .and. y==8  
Print*, x==6 .and. y==8 .or. x==5 .and. y==7
```



```
C:\Users\Spall>a.exe  
T  
T  
F  
T  
C:\Users\Spall>
```

Lecture 9: IF-THEN-ELSE Constructs

Structure:

IF(logical-expression) statement

OR

IF (logical-expression) THEN
 statements

END IF

OR

IF (logical-expression) THEN
 Statements
ELSE
 Statements

END IF

OR

name: IF (logical-expression) THEN
 Statements
ELSE IF (logical-expression) THEN name
 Statements
ELSE IF (logical-expression) THEN name
 Statements

...

ELSE name
 Statements
END IF name

Examples:

```
IF ( x >= 0.0)   y=SQRT (x)
```

OR

```
IF (x>=0.0)   THEN  
    y=SQRT (x)  
END IF
```

Let's look at an example program to solve the quadratic equation.

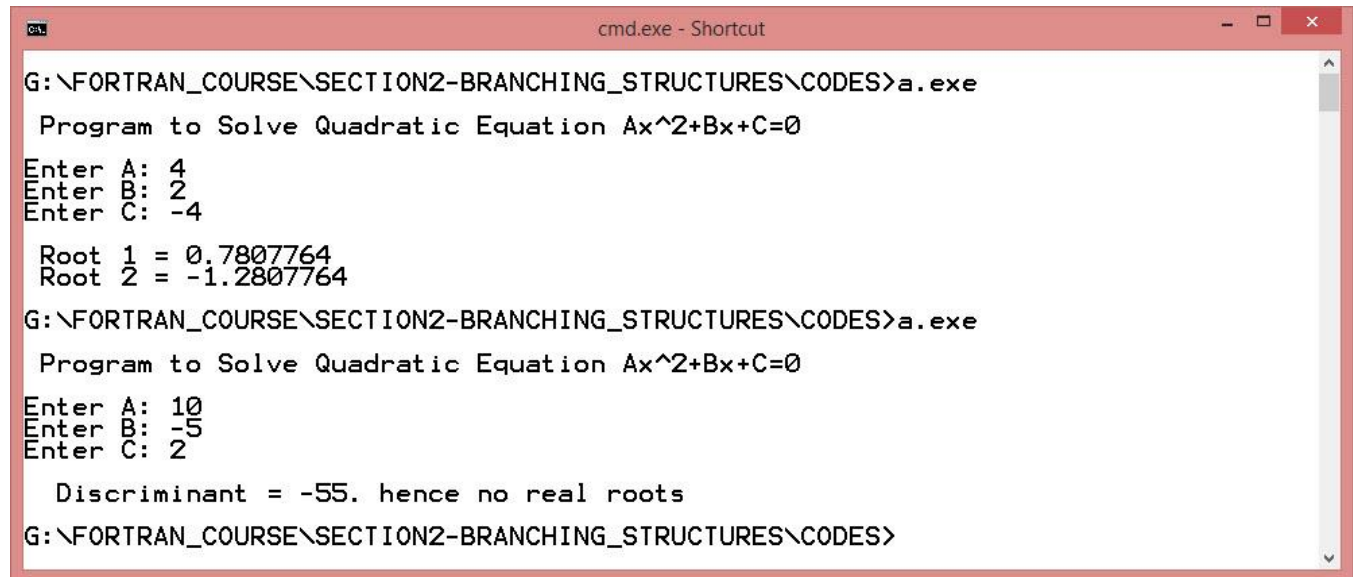
```

!SOLVE THE QUADRATIC EQUATION  $Ax^2+Bx+C=0$ 
PROGRAM quadratic_equation
IMPLICIT none
REAL :: A,B,C,root1,root2,discriminant
PRINT*, 'Program to Solve Quadratic Equation  $Ax^2+Bx+C=0$ '
PRINT*
WRITE(*, '(a)', ADVANCE='NO') 'Enter A: '
READ(*,*) A
WRITE(*, '(a)', ADVANCE='NO') 'Enter B: '
READ(*,*) B
WRITE(*, '(a)', ADVANCE='NO') 'Enter C: '
READ(*,*) C
PRINT*
discriminant=B**2-4.*A*C

IF(discriminant >= 0) THEN                !REAL ROOTS
    root1=(-B+SQRT(discriminant))/(2.*A)
    root2=(-B-SQRT(discriminant))/(2.*A)
    PRINT*, 'Root 1 =', root1
    PRINT*, 'Root 2 =', root2
ELSE
    PRINT*, ' Discriminant =', discriminant, 'hence &
           no real roots'
END IF
END PROGRAM quadratic_equation

```

PROGRAM OUTPUT



```
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>a.exe
  Program to Solve Quadratic Equation  $Ax^2+Bx+C=0$ 
Enter A: 4
Enter B: 2
Enter C: -4
  Root 1 = 0.7807764
  Root 2 = -1.2807764
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>a.exe
  Program to Solve Quadratic Equation  $Ax^2+Bx+C=0$ 
Enter A: 10
Enter B: -5
Enter C: 2
  Discriminant = -55. hence no real roots
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>
```

Lecture 10: Case Construct

Multiple ELSE IF statements can be difficult to follow

The “CASE” construct is a good option.

```
SELECT CASE (selector)
```

```
    CASE(case_selector1)  
        statements
```

```
    CASE(case_selector2)  
        statements
```

```
etc...
```

```
    CASE DEFAULT  
        statements
```

```
END SELECT
```

case-selector is an integer, character, or logical expression, (not real)

case-selector may take the form:

value	if value equals selector
-------	--------------------------

value1 : value2	all values of selector between value1 and value2
-----------------	--

value1 :	all values of selector greater than or equal to value1
----------	--

: value2	all values of selector less than or equal to value2
----------	---

You may also attach a name to the CASE construct:

```
name: SELECT CASE (selector)
      CASE(case_selector1) name
        statements
      CASE(case_selector2) name
        statements
etc...
      CASE DEFAULT name
        statements
END SELECT name
```

EXAMPLE:


```

!PROGRAM TO SORT WIND SPEED USING BEAUFORT SCALE
PROGRAM beaufort_scale
IMPLICIT NONE
REAL :: wind_speed
INTEGER :: integer_wind_speed

WRITE(*, '(a)', ADVANCE='NO') 'Enter the wind speed in Knots:'
READ(*, *) wind_speed
integer_wind_speed = NINT(wind_speed)
PRINT*

SELECT CASE (integer_wind_speed)
    CASE(0)
        PRINT*, 'Force 0; Calm'
    CASE(1:3)
        PRINT*, 'Force 1; Light Air'
    CASE(4:6)
        PRINT*, 'Force 2; Light Breeze'
    CASE(7:10)
        PRINT*, 'Force 3; Gentle Breeze'
    CASE(11:16)
        PRINT*, 'Force 4; Moderate Breeze'
    CASE(17:21)
        PRINT*, 'Force 5; Fresh Breeze'
    CASE(22:27)
        PRINT*, 'Force 6; Strong Breeze'

```

```

CASE(28:33)
    PRINT*, 'Force 7; Near Gale'
CASE(34:40)
    PRINT*, 'Force 8; Gale'
CASE(41:47)
    PRINT*, 'Force 9; Strong Gale'
CASE(48:55)
    PRINT*, 'Force 10; Storm'
CASE(56:63)
    PRINT*, 'Force 11; Violent Storm'
CASE(64:)
    PRINT*, 'Force 12; Hurricane'
END SELECT
END PROGRAM beaufort_scale

```

```

G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>a.exe
Enter the wind speed in Knots: 10.7
    Force 4; Moderate Breeze
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>a.exe
Enter the wind speed in Knots: 77
    Force 12; Hurricane
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>a.exe
Enter the wind speed in Knots: 3.1
    Force 1; Light Air
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>a.exe
Enter the wind speed in Knots: 0.5
    Force 1; Light Air
G:\FORTRAN_COURSE\SECTION2-BRANCHING_STRUCTURES\CODES>

```